

Technical Interview Guide — Snowflake & Data Engineering

Candidate: Nishika Panchal | **Role:** Senior Solution Engineer / Data Engineering (Snowflake focus) | **Round:** Technical Deep Dive

Purpose: Round 1 (Pratik Kulkarni) explicitly deferred all Snowflake/dbt technical validation to this round. The candidate's CV claims specific, senior-level dbt/Snowflake work at IT care Holding AG (May 2025–Mar 2026) — but her German-language CV for the *same role and period* describes only generic Spring Boot/Kubernetes work with no mention of Snowflake or dbt at all. Treat every claim below as **unverified until she demonstrates it in her own words**, ideally with a concrete example, a decision she made, and a trade-off she weighed. Genuine hands-on experience produces specifics unprompted (error messages, exact commands, why-not-X reasoning); rehearsed/documentation-level knowledge stays generic even under follow-up.

Structure of this guide: Questions are now ordered to follow her CV, bullet by bullet, for the IT care Holding AG role — the section headers below quote her CV directly. Work through it top to bottom the same way she describes the role, then use the Miscellaneous section at the end (fundamentals not directly named in a bullet, plus the closing whiteboard scenario) as time allows.

CV Bullet 1

"Designed and implemented a scalable cloud-native data warehousing platform on Snowflake Data Cloud, ingesting large-scale healthcare data (EHR, patient encounters, hospital billing) from AWS S3 data lake with IAM-enforced data governance"

Q1 — Snowflake's architecture

Ask: "Can you describe Snowflake's architecture and how it differs from a traditional on-prem or Hadoop-style data warehouse? Then tell me — at a high level — how you approached 'designing' the platform here. What decisions were actually yours to make?"

Where this is used: Foundational — determines whether she understands *why* Snowflake behaves the way it does (independent scaling, no manual tuning, per-second billing), not just that she's used it, and whether "designed and implemented" reflects real ownership or inherited/copied setup.

Expected answer / keywords to catch:

Three-layer architecture: storage layer (columnar, compressed, stored in cloud object storage — S3/Azure Blob/GCS), compute layer (**virtual warehouses** — independently scalable clusters), and cloud services layer (metadata, query optimization, security, authentication, infrastructure management) Storage and compute are **decoupled** — you can scale one without the other, and multiple warehouses can read the same data simultaneously without contention

Micro-partitions: Snowflake automatically partitions data into 50–500MB compressed micro-partitions, and maintains metadata (min/max values) for pruning — she should mention this enables automatic query pruning without manual partitioning

No indexes to manage, no vacuuming (unlike Redshift) — common differentiator to listen for

On "what was actually hers to design": listen for specifics — warehouse sizing choices, schema/database structure for multi-environment separation, RBAC design — not just "I set it up"

Red flag: If she can only describe warehouses/compute but not the storage/micro-partition side, she may have used Snowflake as a SQL endpoint without understanding the platform.

CV Bullet 2

"Architected end-to-end ELT pipelines using dbt Core (CLI) following the Medallion Framework (Bronze → Silver → Gold) within Snowflake, transforming raw medical records into clean, analytics-ready datasets for BI teams"

Q2 — Medallion architecture in practice

Ask: "Walk me through what actually changes at each layer for one specific dataset — say, patient encounters."

Where used: This is the centerpiece of her claimed role — don't accept a generic definition, push for a concrete walkthrough.

Expected answer / keywords:

Bronze: raw/landed data, minimal transformation, often 1:1 with source (staging models in dbt — `stg_` prefix convention), light typing/renaming only

Silver: cleaned, conformed, deduplicated, business rules applied, joins across sources, still fairly granular

Gold: aggregated, business-ready, modeled for consumption (star schema facts/dimensions or OBT — this is where BI tools query from)

Should be able to name **actual model layers/folders** (dbt convention: `staging/`, `intermediate/`, `marts/`) and map them to Bronze/Silver/Gold naturally

Listen for a **specific transformation decision** — e.g., "we deduplicated on patient_id + encounter_id using a window function in Silver because the source system sent duplicate HL7 messages" — this level of specificity is what separates real experience from a memorized framework

"dbt Core (CLI)" is specifically named — she should be comfortable with CLI commands (`dbt run`, `dbt build`, `dbt test`, `dbt run --select`), not only a GUI/dbt Cloud IDE experience

Follow-up probe: "What would go wrong if you tried to enforce a data quality rule — say, no null patient IDs — in the Bronze layer instead of Silver or Gold?" Tests whether she understands *why* the layering exists, not just its names.

CV Bullet 3

"Developed modular dbt Models with mixed materialisation strategies — Incremental models to process only new/changed records (reducing compute cost) and Ephemeral models as lightweight, reusable intermediate layers"

Q3 — Materialization strategies: incremental vs. ephemeral vs. table vs. view

Ask: "Explain the difference between a table, view, incremental, and ephemeral materialization in dbt. When would you choose each — and specifically, why did you use incremental models for the patient/billing data?"

Expected answer / keywords:

View: no data stored, query run live each time — cheapest storage, most expensive at query time, good for lightweight/low-volume transforms

Table: fully rebuilt (`CREATE OR REPLACE`) on every run — simple, but expensive/slow for large datasets since everything is recomputed

Incremental: only processes new/changed rows since the last run, using an `is_incremental()` macro block and a filter like `WHERE updated_at > (SELECT MAX(updated_at) FROM {{ this }})` — she should mention the

`unique_key` config for merge/upsert behavior and dbt's incremental strategies (`merge`, `delete+insert`, `append`) — on Snowflake specifically, `merge` is the default/common strategy

Ephemeral: not materialized as a table/view at all — inlined as a CTE into downstream models at compile time; no storage cost, but can't be queried directly outside dbt and can't be reused across many downstream models without recompiling the same logic repeatedly (a trade-off she should be aware of) Should articulate the **actual reason** she chose incremental for patient/billing data: large volume + slow-changing = full rebuild wasteful; ephemeral for something like reusable lightweight staging logic that's cheap to inline

Follow-up probe: "What happens if your incremental model's source data gets a late-arriving update to a row older than your incremental filter watermark?" Classic incremental-model gotcha (late-arriving/backfilled data silently missed unless handled via a lookback window or periodic full-refresh) — a candidate with real production experience has usually hit this.

Follow-up probe 2: "How do you force a full rebuild of an incremental model when needed?" — expected: `dbt run --full-refresh`.

CV Bullet 4

"Built fully metadata-driven, parameterizable pipelines using advanced Jinja Templates and Macros, eliminating repetitive SQL logic and reducing code volume by ~40%"

Q4 — Jinja templating and macros

Ask: "Give me a concrete example of a macro you wrote and what repetitive SQL it replaced. How did you actually arrive at the ~40% number?"

Where used: Tests the specific, quantified claim on her CV directly — quantified claims like "~40%" are exactly what to probe on measurement, not just mechanism.

Expected answer / keywords:

A **real macro pattern** — common examples: a macro generating column lists dynamically from `information_schema`, a macro for standard SCD/audit columns (`_loaded_at`, `_source_system`) injected into every model, a macro generating repetitive `CASE WHEN`/pivot logic parametrically, or a macro wrapping a common test pattern

Jinja control structures: `{% for %}`, `{% if %}`, `{% set %}`, `{{ }}` for expression substitution

Distinguishes **macros** (reusable SQL-generating functions, `.sql` files in `macros/`) from **packages** (e.g., `dbt_utils` — ask if she used any, like `dbt_utils.generate_surrogate_key` or `dbt_utils.pivot`)

The "40% code reduction" claim should have a believable origin story — e.g., "we had 15 near-identical staging models and templated the common structure" — not just an abstract number

Red flag: If she can only describe Jinja in the abstract ("it lets you template SQL") without a concrete macro example, this specific claim is likely embellished or borrowed from someone else's work.

CV Bullet 5

"Deployed dbt Snapshots (SCD Type 2) on critical patient health metrics and billing dimensions to maintain complete historical audit trails — preserving both old and new record states on every change"

Q5 — dbt Snapshots and SCD Type 2

Ask: "Explain what SCD Type 2 is, how dbt snapshots implement it mechanically, and why it mattered for this use case."

Where used: A well-defined dbt feature with a precise mechanism — hard to fake convincingly.

Expected answer / keywords:

SCD Type 2 = Slowly Changing Dimension, Type 2: instead of overwriting a changed attribute, insert a new row and preserve history — every version of a record is retained with validity windows

dbt snapshot mechanics: adds `dbt_valid_from` and `dbt_valid_to` columns (and a `dbt_scd_id` surrogate key); when a tracked column changes, the old row gets `dbt_valid_to` set to the current timestamp and a new row is inserted with `dbt_valid_to = NULL` (current/active record)

Two snapshot strategies: `timestamp` (uses an `updated_at` column on the source) vs. `check` (compares specified columns for changes when no reliable `updated_at` exists) — she should know which one she used and why

Should articulate the *business* reason: audit trail requirement for healthcare data (e.g., "what was the patient's billing status as of a specific date" for compliance/reporting)

Snapshots run on a schedule (separate from regular model runs) — `dbt snapshot` command

Follow-up probe: "Where do snapshots physically live, and can you query historical state directly?"

Expected: snapshots are materialized as physical tables in a snapshot schema; yes, query `WHERE dbt_valid_to IS NULL` for current state or filter by date for a point-in-time view.

CV Bullet 6

"Enforced production-grade data quality using automated dbt Singular Tests and schema assertions with severity flags (Warnings and Errors), blocking data anomalies such as duplicate Patient IDs and null clinical fields before deployment"

Q6 — dbt Testing (Singular tests & schema tests)

Ask: "Walk me through how you'd actually write one of these — a test for duplicate Patient IDs, for example."

Expected answer / keywords:

Schema tests (generic tests, defined in YAML): built-in ones are `unique`, `not_null`, `accepted_values`, `relationships` — she should describe writing `not_null` + `unique` on a `patient_id` column directly in `schema.yml`

Singular tests: a `.sql` file in the `tests/` folder that returns rows that *fail* the test — dbt considers the test failed if the query returns any rows. She should describe writing a custom query (e.g., detecting duplicate patient IDs via `GROUP BY ... HAVING COUNT(*) > 1`)

Severity flags: `severity: warn` VS `severity: error` in the test config — warn logs but doesn't fail the build/CI pipeline, error blocks it. She should explain the judgment call of which checks are "blocking" vs "advisory" — e.g., a null in a non-critical field might warn, while a duplicate patient ID (patient safety/billing risk) should error and block deployment

Bonus: mentions `dbt test`, or `dbt build` (runs models and tests together), or dbt's test result storage for auditability

Follow-up probe: "Say a singular test starts failing in production because of a genuine upstream data issue, not a bug in your model. What's your process from alert to resolution?" Tests operational maturity, not just syntax.

CV Bullet 7

"Applied Dimensional Modelling (Star Schema — Fact & Dimension tables) and One Big Table (OBT) patterns in the Gold layer, enabling BI teams to analyse patient readmission rates and operational KPIs"

Q7a — Star Schema and Dimensional Modeling

Ask: "Design one with me: what would the fact table and dimensions look like for analyzing patient readmission rates?"

Where used: Whiteboard/verbal design exercise — best way to distinguish genuine modeling skill from buzzword familiarity.

Expected answer / keywords:

Fact table: grain should be explicitly stated (e.g., "one row per patient encounter/admission") containing **foreign keys** to dimensions plus **measures** (numeric, additive values — length of stay, billed amount, readmission flag)

Dimensions: `dim_patient`, `dim_provider`, `dim_date` (classic date dimension), `dim_diagnosis`, etc. — descriptive attributes, typically denormalized

Grain should come up explicitly — the single most important concept in dimensional modeling; if she never says "grain" (or an equivalent) unprompted, probe for it directly

Bonus: distinguishes **additive, semi-additive (e.g., balances), and non-additive measures**

Should know **surrogate keys** vs natural/business keys, and why surrogates are used (stability across SCD changes, performance)

Follow-up probe: "How would readmission specifically be modeled — as a flag on the fact table, or would you need a self-join or a separate logic layer to identify it?" (Readmission is typically derived — e.g., "same patient admitted again within 30 days of discharge" — requiring a self-referencing lookback; nice test of whether she thinks through derived metrics or just lists dimensions.)

Q7b — OBT (One Big Table) vs. Star Schema — when to use which

Ask: "When would you choose one over the other, and did you use both for different consumers?"

Expected answer / keywords:

OBT: denormalized, wide table with dimensions flattened in — optimized for simplicity and query performance for BI tools that don't handle joins well (or for self-service/less technical analysts), at the cost of storage and potential redundancy

Star schema: normalized dimensions, better for complex/flexible analytical querying, easier to maintain consistency across many facts sharing dimensions

A thoughtful answer notes that on Snowflake specifically, storage is cheap and compute/joins are fast, so OBT's traditional "avoid joins" rationale is less critical than on legacy warehouses — nuance to listen for
Genuine experience: a specific consumer example (e.g., "the readmission dashboard team wanted OBT for simplicity, but the finance team's Star Schema queries needed the flexibility to join billing facts against multiple dimension sets")

CV Bullet 8

"Wrote Python scripts using boto3 to automate AWS S3 data ingestion workflows — handling file uploads, bucket triggers, and pre-processing before loading into the Snowflake data warehouse"

Q8 — S3 ingestion with boto3

Ask: "Describe the actual flow: what triggers your script, what does it do, and how does the data end up in Snowflake?"

Expected answer / keywords:

boto3 = AWS SDK for Python; she should describe either a **Lambda function triggered by an S3 event** (`s3:ObjectCreated:*`) or a scheduled/polling script

Pre-processing examples: file validation, schema checks, format conversion (e.g., renaming/partitioning files, converting to Parquet), before landing in a "processed" bucket/prefix that Snowflake's stage points to

Should connect this back to Snowflake concretely — either triggering **Snowpipe** via SQS notification, or calling a Snowflake `COPY INTO` / running a Task

IAM: least-privilege roles/policies for the Lambda or script to access the specific bucket, not broad credentials — this is also where **Bullet 1's "IAM-enforced data governance"** should get concretely substantiated

Follow-up probe: "What did you do about error handling — a malformed file landing in the bucket?" Real-world experience surfaces here (dead-letter handling, moving bad files to a quarantine prefix, alerting) — a fabricated answer tends to stop at the happy path.

CV Bullet 9

"Configured Snowflake File Formats and Stages to ensure high-speed, cost-efficient loading of sensitive patient records into the data warehouse"

Q9 — Stages and File Formats

Ask: "Walk me through how you'd load a batch of CSV or JSON files landing in an S3 bucket into a Snowflake table. What Snowflake objects are involved?"

Expected answer / keywords:

External stage pointing to the S3 bucket (created with a **storage integration** for IAM-based auth — more secure than embedding keys; should mention this if her AWS IAM understanding is solid)

File Format object defining CSV/JSON parsing rules (delimiter, header skipping, compression,

`STRIP_OUTER_ARRAY` for JSON, error handling like `ON_ERROR = 'CONTINUE'` or `'SKIP_FILE'`)

COPY INTO command to bulk load from stage to table

For continuous/near-real-time ingestion: **Snowpipe** (auto-ingest via S3 event notifications/SQS) vs. scheduled `COPY INTO` via a **Task**

Should distinguish bulk loading (`COPY INTO`, warehouse-billed) from Snowpipe (serverless, billed per-second on actual usage)

"Sensitive patient records" — listen for any mention of masking policies or row access policies, relevant given healthcare data sensitivity (not required, but a strong positive signal)

Follow-up probe: "Did you use Snowpipe or scheduled COPY INTO?" — if she claims real-time ingestion (see Bullet 11), ask specifically how the S3 event → Snowpipe trigger was wired, or whether she means Kafka → Snowflake via the Kafka connector instead. These are different mechanisms and she should know which one she actually built.

CV Bullet 10

"Integrated AWS cloud infrastructure (S3, IAM, EC2, CloudWatch) with Snowflake and managed multi-environment CI/CD deployments (Dev → QA → Prod) using Git branching and GitHub Actions"

Q10 — CI/CD for data pipelines (multi-environment dbt deployment)

Ask: "Walk me through what happens from a merged PR to a production deployment."

Expected answer / keywords:

Environment separation typically via **dbt targets/profiles** (different Snowflake databases/schemas or even separate accounts per environment) and **dbt Cloud environments** or CLI `--target` flag

GitHub Actions workflow: triggered on PR (run `dbt build`/`dbt test` against a Dev/CI schema — sometimes using **slim CI** / `dbt build --select state:modified+` to only run affected models, a good sign of real maturity if she mentions it) → merge to main → deploy to QA → promote to Prod, often with manual approval gates
Should mention **dbt state/artifacts** (`manifest.json`) for detecting what changed, if at a sophisticated level
Secrets/credentials handled via GitHub Secrets, not hardcoded

CloudWatch: should mention what she actually monitored — pipeline failures, Lambda errors, custom metrics/alarms — not just name-drop it

Red flag: Generic "we used GitHub Actions for CI/CD" without being able to describe environment promotion mechanics specific to dbt (targets, schemas, state) suggests she may have used a pre-built pipeline someone else set up.

CV Bullet 11

"Designed and operated real-time streaming ingestion pipelines alongside batch ELT workflows, using containerised microservices (Docker, Kubernetes) and event-driven messaging (Apache Kafka)"

Q11 — Kafka + Snowflake: real-time vs. batch

Ask: "How does Kafka data actually get into Snowflake, and how did you reconcile a real-time stream with a batch dbt model that might run hourly?"

Where used: The highest-value question in the whole guide — it directly tests the most ambitious and most CV-only claim, and exposes a real architectural tension (streaming source, batch transformation) that only someone who's actually built this would have had to solve.

Expected answer / keywords:

Mechanism: **Snowflake Connector for Kafka** — consumes topics and loads into Snowflake tables (internally uses Snowpipe under the hood), or alternatively Kafka → S3 (via Kafka Connect S3 sink) → Snowpipe

Should acknowledge the **reconciliation problem** explicitly: raw streaming data lands continuously, but Bronze→Silver→Gold dbt transformations run on a schedule (batch) — meaning "real-time" typically only extends to the *raw ingestion* layer, not the full pipeline, unless she specifically used **Dynamic Tables** or **Streams+Tasks** for lower-latency transformation

A strong/honest answer draws this distinction unprompted; a weaker one conflates "we used Kafka" with "the whole pipeline was real-time"

Bonus: mentions **Dynamic Tables** (Snowflake's newer declarative incremental materialization with a target lag) as a way to bridge streaming ingestion with near-real-time transformed output — fairly recent feature, don't require it, but a strong positive signal if she brings it up

"Containerised microservices (Docker, Kubernetes)" — ask what specifically ran in containers here (a Kafka consumer? the boto3 pre-processing scripts?) — should have a concrete answer, not just a skills-list echo

Follow-up probe: "What was actually streamed through Kafka in this healthcare context — what's the event?" Push for a concrete example (e.g., "new lab result posted," "encounter status change"). Vague or evasive answers here are the biggest red flag in this entire guide, since this bullet is the most technically demanding claim on her CV and the easiest to have added without hands-on experience.

Miscellaneous — Not Tied to a Specific CV Bullet

These come from her skills list rather than a named accomplishment, or are designed as a general closing exercise. Use as time allows, or as tiebreakers if you're still unsure after the bullet-by-bullet questions above.

M1 — Virtual warehouses and cost/performance

Ask: "How would you size and manage virtual warehouses for a mixed workload — say, heavy ELT batch loads at night and ad-hoc BI queries during the day?"

Expected answer / keywords:

Separate warehouses per workload type (e.g., `ETL_WH` vs `BI_WH`) so they don't compete for resources and can be sized/scaled independently

Auto-suspend / auto-resume to avoid paying for idle compute

Multi-cluster warehouses for concurrency scaling vs. **warehouse size (T-shirt sizing, XS-6XL)** for single-query performance

Query Profile / Query History to identify undersized (spilling to disk/remote) or oversized (low utilization) warehouses

Bonus: **resource monitors** to cap credit spend

Follow-up probe: "You mentioned reducing compute cost with incremental models — how did you actually measure that the change reduced cost?" Genuine experience → Query History, credits consumed, or `SNOWFLAKE.ACCOUNT_USAGE` views.

M2 — Streams and Tasks

Ask: "Her skills list mentions Streams and Tasks — how have you used them, and what problem do they solve versus just re-running a full COPY/MERGE?"

Expected answer / keywords:

Stream = Snowflake's native **CDC (change data capture)** object — tracks inserts/updates/deletes on a table since the last consumption, using metadata columns (`METADATA$ACTION`, `METADATA$ISUPDATE`, `METADATA$ROW_ID`)

Task = scheduled or triggered SQL execution — often chained with a stream ("if stream has data, run this MERGE")

Snowflake's lightweight alternative to needing an external orchestrator (Airflow) for simple incremental pipelines

Bonus: `SYSTEM$STREAM_HAS_DATA()` in task conditionals, or task DAGs (`AFTER` dependency chains)

Red flag: These appear only in her skills list, not in her CV bullets — a good "does she actually know this or did she just list it" check.

M3 — End-to-end whiteboard scenario (closing exercise)

Ask: "Forget your CV for a second — here's a fresh scenario. A retail client sends daily CSV files of transactions to an S3 bucket. They want a Snowflake-based pipeline producing a daily sales-by-region dashboard, with the ability to see how a customer's loyalty tier changed over time. Design the pipeline end-to-end: ingestion, transformation layers, modeling, and how you'd handle the loyalty tier history requirement specifically."

Where used: The best single question in this guide — a novel scenario she can't have memorized, requiring synthesis of everything above. Gives a clean signal independent of anything she prepared.

Expected answer, roughly:

Ingestion: External stage over S3 + File Format for CSV, `COPY INTO` a raw/Bronze table (or Snowpipe if she justifies near-real-time, though daily batch doesn't need it — good if she notices this and doesn't over-engineer)

Bronze → Silver: dbt staging models cleaning types, deduplicating, standardizing

Loyalty tier history → should trigger "SCD Type 2 / dbt snapshot" in her head unprompted — the key test; if she doesn't independently connect this requirement to snapshots without you naming it, that's a meaningful signal about depth vs. rote recall from her own CV bullets

Gold layer: `fct_transactions` fact table (grain: one row per transaction) + `dim_customer` (with SCD2 loyalty tier tracking), `dim_region`, `dim_date` — Star Schema for the dashboard

Bonus: testing (uniqueness/not-null on keys), incremental materialization for the daily fact table given growing volume, orchestration (Task or external scheduler like Airflow/dbt Cloud scheduler)

What to weight most heavily: Whether she reaches for **snapshots/SCD2 on her own** when she hears "how a customer's tier changed over time" — the cleanest test in the whole interview of whether her CV reflects real judgment or copied vocabulary.

General Interview Process Notes

Sequencing: Following her CV bullet-by-bullet (as structured above) keeps the conversation natural and lets you watch how she narrates her own experience in order. If she stumbles early (Bullet 1 / architecture fundamentals), weight the rest accordingly and consider spending less time on the more advanced dbt-specific bullets.

Weight Bullets 4, 5, and 11 (Q4, Q5, Q11) most heavily — these map to the most specific, quantified, and technically demanding claims on her CV (40% code reduction via macros, SCD Type 2 snapshots, real-time Kafka streaming). If she's genuine, these should be her strongest, most detailed answers, not her weakest.

Use M3 (whiteboard scenario) as the tiebreaker if you're still unsure after working through the CV — a novel scenario is the hardest thing to fake and the clearest signal of applied judgment vs. memorized/borrowed content.

On the German-language claim: Round 1 noted her previous project was "completely in German" but flagged her German skills for testing. Consider asking her to describe one technical concept (e.g., explain incremental models) briefly in German — both to validate the language claim and, indirectly, to see if her German-language project experience narrative holds up against the German CV's different job description for the same period.

Scoring: Given Deloitte's 1-5 RISE scale used in the Round 1 feedback doc, I'd suggest anchoring: 1-2 if she can't get past Bullet 1/architecture fundamentals; 3 if she handles the general dbt/Snowflake concepts well but goes vague/generic on the specific claims in Bullets 4, 5, and 11; 4-5 only if she demonstrates those specific claims with concrete, self-generated detail (actual macro examples, the incremental-watermark gotcha, the streaming/batch reconciliation problem) without you having to lead her there.